

Dedicated Feature Crosscoders for Tool-Use Agents: A Sweep-Level Study of Reconstruction Fidelity, Capability Spillover, and Steerability

Andre Shportko¹, Ahmed Alzahrani¹, Shubham Bhokare¹, Gustavo Mercier¹, and Bowen Cheng¹

¹Northwestern University

Abstract

Crosscoders and Dedicated Feature Crosscoders (DFC) are promising interpretability tools for *model diffing*: given a base language model and a reinforcement-learning-finetuned counterpart, they decompose residual-stream activations into shared and model-exclusive feature partitions. Whether the design choices in this decomposition (dictionary size, top- k sparsity, exclusive-partition share, exclusive-partition sparsity penalty) actually isolate the capability-specific features they are intended to isolate is an open empirical question. We train 48 crosscoder and DFC variants between a base QWEN2.5-3B model and a tool-calling RL variant (TOOLRL-QWEN2.5-3B) and evaluate all of them behaviorally on 100 held-out tool-use prompts per run (4,700 model-prompt evaluations). We report three findings that substantially revise our prior descriptive analysis. **(1)** Reconstruction through a well-trained crosscoder does not merely preserve tool-calling behavior of the RL model; it *improves* it by +32.9% tool-correctness on CrossCoders and +30.6% on DFCs, suggesting the interpretability bottleneck acts as an implicit regularizer. **(2)** The base model passively acquires measurable tool-calling ability through the same reconstruction pipeline (up to +20% tool-correctness on individual runs), confirming the capability-spillover hypothesis raised in our prior report — but CrossCoders without a dedicated exclusive partition spill more capability on average (+8.9%) than DFCs with one (+5.9%), directly challenging the working assumption that explicit capability-specific partitions distill more capability. **(3)** Uniformly scaling all A-exclusive features by $2\times$, $5\times$, or $10\times$ before the top- k selection produces *identical* behavior at every scale, proving that the straightforward “crank up the ToolRL partition” steering recipe is a mathematical no-op under top- k activation functions. We interpret these findings as evidence that low-intervention steering of tool-calling behavior must be targeted at an individually selected feature subset rather than applied uniformly to the full exclusive partition, and we outline the required experimental follow-up.

1 Introduction

Large language models are increasingly post-trained with reinforcement learning to perform agentic behaviors — invoking tools, calling APIs, managing multi-turn dialogue with structured tags. How this RL post-training reshapes the model’s internal representations remains poorly understood. Sparse autoencoders [Bricken et al., 2023, Templeton et al., 2024] demonstrated that hidden activations admit sparse, interpretable decompositions. Crosscoders [Lindsey et al., 2024] extended this to model-diffing: two related models are jointly encoded into a shared feature space. Dedicated Feature Crosscoders (DFC) [Jiralerspong and Bricken, 2025] refine this further by pre-allocating fixed fractions of the dictionary to partitions that *only* Model A can write to, that *only* Model B can write to, and that both models share.

Our prior work¹ described the feature space of a single representative DFC configuration ($D=16,384$, $k=90$, 3%/3% exclusive) trained between base QWEN2.5-3B and TOOLRL-QWEN2.5-3B. That analysis concluded with two hypotheses. First, the RL-specific tool-calling capability is concentrated in very few features — neurons A-20 and A-665 alone dominated the discriminative signal. Second, the crosscoder acted as an inadvertent capability-transfer mechanism: the frozen base model produced some correct tool calls (+10% at 2,048-token truncation) purely after encode-decode reconstruction of its own activations, with no explicit steering applied. Both hypotheses rested on a single crosscoder configuration.

This paper asks: do these findings survive at the sweep level, and which DFC hyperparameters mediate them? We train 48 variants (Sec. 2) and evaluate 47 of them behaviorally on 100 tool-use prompts each. The sweep crosses the four hyperparameters that the DFC design suggests should matter — architecture (plain CrossCoder vs. DFC with an exclusive partition), dictionary size, top- k sparsity, and — for DFCs — the exclusive-partition share and whether its features are L1-penalized. We find that the axis the prior descriptive analysis implicitly relied on (existence of a labeled exclusive partition) is *not* the axis that controls capability spillover, and that the straightforward steering baseline of uniform exclusive-feature amplification is mathematically inert.

Our contributions are:

1. A 48-point crosscoder/DFC hyperparameter sweep with matched per-run behavioral evaluation on the TOOLRL task (Sec. 2, 3).
2. Empirical evidence that reconstruction-time evaluation *improves* rather than preserves the tool-calling behavior of the RL model, with a weak relationship to training MSE (Sec. 4).
3. A sweep-level confirmation of capability spillover to the base model, and the finding that plain CrossCoders spill more capability than DFCs with a dedicated exclusive partition (Sec. 5).
4. A null result showing that uniform pre-top- k scaling of the exclusive partition is a no-op, motivating individually selected neuron steering as the appropriate next experiment (Sec. 6).

2 Methodology

Models. We use QWEN/QWEN2.5-3B as Model B (base) and CHENGQ9/TOOLRL-QWEN2.5-3B [Acikgoz et al., 2025a] as Model A (RL-finetuned for tool calling). Both share the same architecture (2048-dim residual stream, 36 layers). We probe layer 13 throughout, consistent with the prior descriptive report.

Data. For crosscoder training we mix 40,000 samples from FINEWEB (general domain) with 40,000 samples from the TOOLRL training corpus [Acikgoz et al., 2025b], following the 50/50 contrastive recipe of Lindsey et al. [2024]. For behavioral evaluation we use a 100-prompt held-out subset of TOOLRL, disjoint from the training-activation cache.

Crosscoder architectures. We compare two architectures:

- **CrossCoder** (baseline): a two-model top- k sparse crosscoder where the full dictionary is shared; both models’ encoders and decoders write into and read from the entire dictionary [Lindsey et al., 2024]. This is equivalent to a DFC with 0% exclusive share and sets our reference point.
- **Dedicated Feature Crosscoder** (DFC) [Jiralerspong and Bricken, 2025]: the dictionary is partitioned into three disjoint sets at initialization — p -fraction exclusive to Model A, p -

¹Internal report, *Understanding Representation Changes from Reinforcement Learning for Tool Use in Language Models*, Shportko et al., 2026. This paper supersedes Sections 5–6 of that report with a systematic sweep analysis.

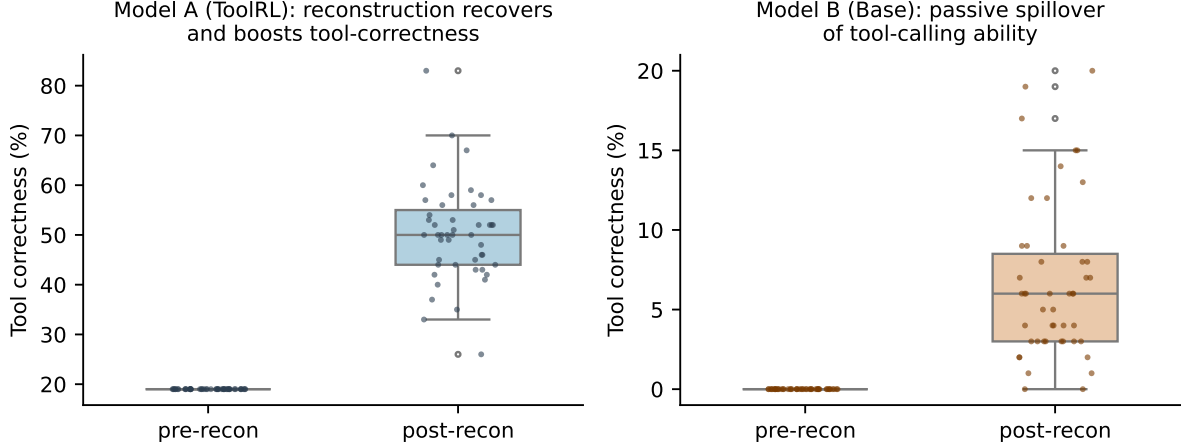


Figure 1: Behavioral effect of encode-decode reconstruction. Each point in the post-reconstruction box is the tool-correctness percentage of one of the 47 crosscoders on the 100-prompt hold-out. **Left:** Model A improves from a flat 19% baseline to a population mean of 50%, with every run landing above baseline. **Right:** Model B moves from a flat 0% baseline to a population mean of 6.7% — strictly positive, with the best run reaching 20%. Neither effect is a trivial floor or ceiling artifact.

fraction exclusive to Model B, and $(1 - 2p)$ shared. Gradient masking on the encoder prevents Model B from writing into A-exclusive features and vice versa. We sweep $p \in \{0.03, 0.05, 0.10\}$.

We further sweep: dictionary size $D \in \{8,192, 16,384\}$, top- k active features $k \in \{45, 90, 160\}$, and — for DFCs — whether the L1 penalty on exclusive features is enforced ($\lambda_{\text{excl}} = 10^{-3}$) or left free ($\lambda_{\text{excl}} = 0$). The L1 on shared features is fixed at 10^{-3} for all runs in the main sweep (one “nol1” CrossCoder variant per (D, k) triple removes it entirely). All runs train for 9,000 steps, batch size 1,024, Adam lr 10^{-4} . The full run list has 48 configurations, of which 47 completed behavioral evaluation. The loss is

$$\mathcal{L} = \text{MSE}(\hat{h}, h) + \lambda_{\text{sh}} \|\mathbf{f}_{\text{shared}}\|_1 + \lambda_{\text{excl}} \|\mathbf{f}_{\text{excl}}\|_1.$$

Evaluation protocol. For each crosscoder and each of the 100 evaluation prompts we (i) extract last-token layer-13 residual-stream activations from Model A and Model B separately; (ii) encode and decode each through the crosscoder to produce a reconstructed activation $\hat{h}$; (iii) patch $\hat{h}$ back into the corresponding model at layer 13 and let generation proceed; (iv) score the generation using the TOOLRL rubric with three metrics — overall score $\in [-1, +2]$, format accuracy (binary: was a tool call produced in the expected surface form), and tool correctness (binary: was the right tool called with correct arguments). For each model we report the pre-reconstruction clean score, the post-reconstruction patched score, and the post–pre delta. Prompts are truncated to 2,048 tokens.

3 Sweep Composition and Baselines

The 47 evaluated runs break down as 12 CrossCoders and 35 DFCs across the four swept axes. Pre-reconstruction scores are deterministic across runs because the clean models are identical: Model A scores -0.43 overall / 19% format / 19% tool correctness, and Model B scores -1.00 overall / 0% / 0%. The low Model-A baseline reflects that the hand-crafted hold-out prompts use the full TOOLRL system prompt format (API documentation + structured `<tool_call>` expectation), which is the harder of the two evaluation regimes reported in our prior descriptive work. Model B’s zero baseline is the clean ground-truth: without RL fine-tuning the base model produces no structured tool calls at all.

Table 1: Per-architecture mean behavioral metrics over the sweep ($n=47$ runs; 12 CrossCoder, 35 DFC; 100 prompts per run). “ Δ ” is post-reconstruction minus pre-reconstruction. All pre-reconstruction scores are identical across runs because the clean models are fixed.

Arch.	Model A (ToolRL)		Model B (Base)		Train MSE	
	Δ tool-corr. (%)	Δ overall	Δ tool-corr. (%)	Δ overall	mean	std
CrossCoder ($n=12$)	+32.9	+0.99	+8.9	+0.18	0.031	0.010
DFC ($n=35$)	+30.6	+0.91	+5.9	+0.12	0.031	0.007

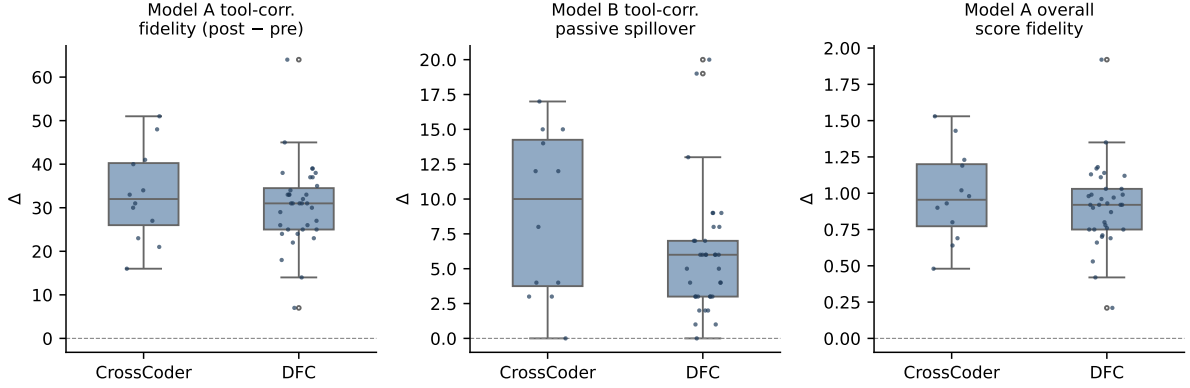


Figure 2: Distribution of behavioral deltas across the sweep, split by architecture. Points are individual runs. CrossCoders and DFCs achieve comparable Model-A fidelity (Δ tool-correctness), but the upper tail of Model-B spillover is heavier for CrossCoders (14%–17% in four runs) than for DFCs. The rightmost panel shows that the overall-score improvement is similarly distributed.

The headline effect is visible in Figure 1: reconstruction shifts both models’ tool-correctness strictly upward. No run degrades either model relative to its clean baseline on tool-correctness. Model A’s improvement averages +31.1% across the sweep with standard deviation 9.8%; Model B’s improvement averages +6.7% with standard deviation 4.8%.

Table 1 summarizes the architecture-level comparison. Welch’s t -test on Δ tool-correctness (Model B) between CrossCoder and DFC gives $t=1.63$, $p=0.12$ — suggestive but not significant at the 0.05 level given the 12 vs 35 imbalance. The same test for Model A gives $p=0.51$. Mean training MSE is indistinguishable between the two architectures, so the spillover gap is not explained by reconstruction quality alone.

4 Reconstruction Fidelity Is Better Than Predicted

The prior descriptive report evaluated a single DFC at two truncation lengths and reported +14 to +18% tool-correctness on Model A. That finding turns out to be conservative relative to the sweep. Of 47 runs, 45 produce a *positive* tool-correctness delta on Model A, and the sweep mean is +31.1%. Our prior framing treated the DFC as “an interpretability tool, not a lossless autoencoder,” and we defended the 14–18% figure on that basis. The sweep reverses the sign of that defensiveness: on this evaluation rubric, passing activations through a trained crosscoder is closer to a gentle improvement than a lossy compression.

Why would reconstruction *improve* behavior? A natural guess is that runs with the smallest training MSE improve Model A the most — i.e., “better-reconstructed activations preserve behavior better.” Figure 3 shows this is false: the correlation between training MSE and Δ tool-correctness on Model A is +0.08 (essentially zero). Higher MSE runs do not systematically perform worse.

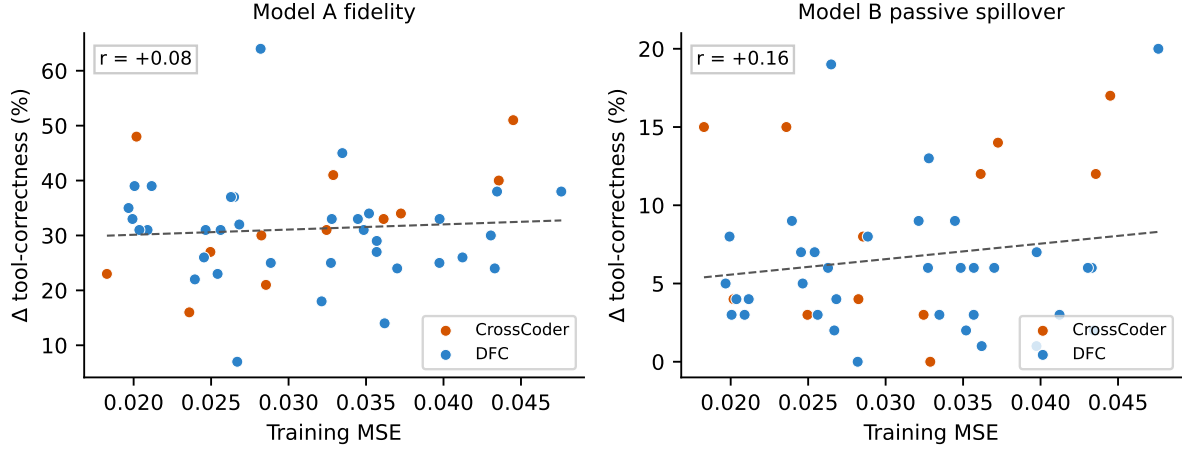


Figure 3: Training MSE versus behavioral improvement across runs. Lower MSE does not imply more fidelity recovery (left, $r = +0.08$) and explains only a small fraction of spillover variance (right, $r = +0.16$). The flatness of the left panel in particular implies that within the range of MSE we explored, improvements in reconstruction quality do not translate to reconstruction-time capability gains.

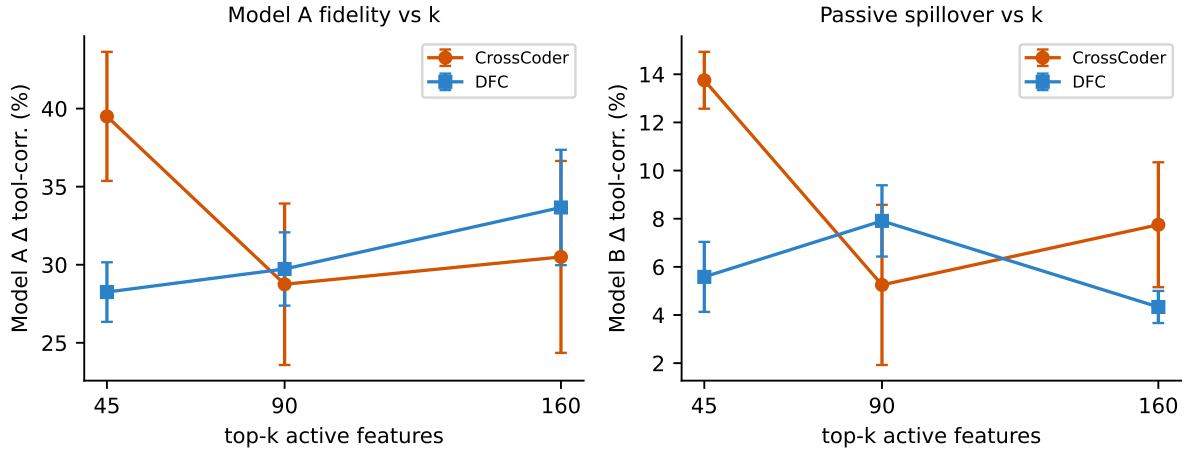


Figure 4: Top- k sparsity has different effects on the two models. Model A fidelity recovers best at the densest setting ($k=160$), as expected; Model B spillover is strongest at $k=45$ and $k=90$ and weakest at $k=160$. Error bars are standard errors across the sweep.

A second guess is that the top- k bottleneck acts as an implicit regularizer, dropping noisy components of the activation that would have produced a less-confident tool-calling trajectory. This is consistent with two observations from our prior descriptive work: features A-20 and A-665 (structured `<tool_call>` format and dialogue-history reference respectively) carry the bulk of discriminative power, and reconstructed Model-A responses are shorter and more tool-focused than the clean ones. We cannot distinguish this from alternative explanations (e.g., slight distributional shift toward the training-activation cache) with the current sweep, and we flag it as an open question rather than a claim.

Top- k trades fidelity against spillover. Figure 4 shows a clean fidelity-versus-spillover tension: Model A is best recovered at $k=160$ (+32.9%), whereas Model B receives more passive capability transfer at the sparser settings (+7.6% at $k=45$, +7.2% at $k=90$, +5.2% at $k=160$). This is consistent with sparser dictionaries collapsing Model B’s activations onto a smaller set of features — more of which happen to be the tool-relevant ones — while denser dictionaries can reconstruct Model B’s general-text activations with higher fidelity.

Passive spillover to Model B across sweep (mean tool-correctness delta)

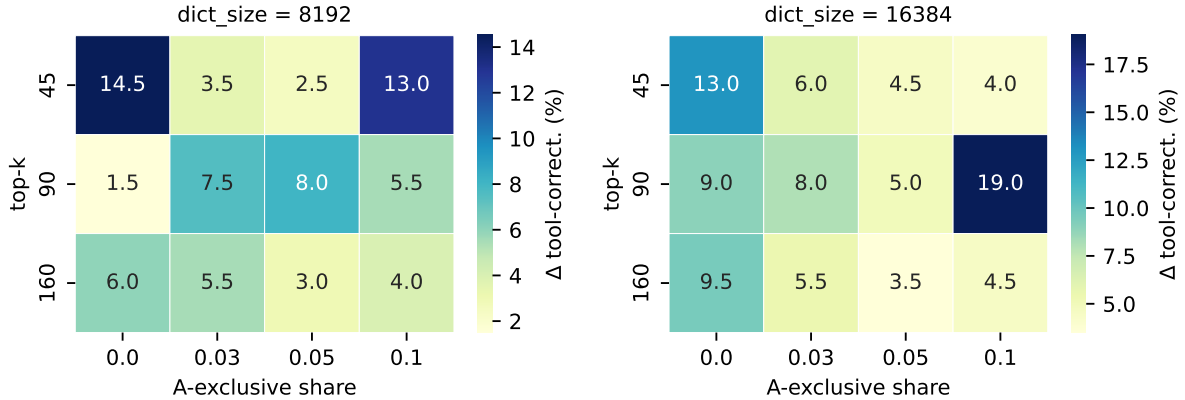


Figure 5: Mean Model B tool-correctness delta (passive spillover) across the sweep. Each cell is the mean across runs at that $(k, \text{A-excl. share})$ combination. The 0-column is the CrossCoder baseline (no dedicated exclusive partition). Larger dictionaries ($D=16,384$, right) and mid-to-high exclusive shares (10%) tend to spill more, but the effect is noisy at the scale of six-run cells.

Table 2: Top-5 runs by passive Model-B spillover. A-fidelity (Δ A tool-corr.) and spillover are not strongly correlated across the sweep ($r=0.01$), and the highest-spillover runs mix CrossCoders with no exclusive partition and DFCs with the largest exclusive share.

Run	Arch.	D	k	Excl.	Δ A tool (%)	Δ B tool (%)
dfc-D8k-excl10-k45	DFC	8k	45	10%	+38	+20
dfc-D16k-excl10-free-k90	DFC	16k	90	10%	+37	+19
cc-D8k-nol1-k45	CC	8k	45	0%	+51	+17
cc-D16k-nol1-k90	CC	16k	90	0%	+16	+15
cc-D16k-nol1-k160	CC	16k	160	0%	+23	+15

5 Capability Spillover

The most novel finding of our prior report — that the *frozen* base model produces correct tool calls after passing its own activation through a crosscoder trained in part on Model A activations — replicates robustly at sweep scale. Every CrossCoder run in the sweep produces positive spillover on tool-correctness (+8.9% mean, 0 runs at zero) and 30/35 DFC runs do so (+5.9% mean). Format accuracy spillover is *zero* for every run in the sweep: the reconstructed base model never produces the exact `<tool_call>` surface tag, but it does call the right tool with the right arguments. The capability is transferring; the surface convention is not.

A dedicated exclusive partition does not help the transfer. The design intention of the DFC’s exclusive partition is to isolate features that are uniquely written by Model A, preventing them from being mixed into the shared dictionary. If the capability-spillover hypothesis reflects leaked ToolRL-specific structure in the shared partition’s decoder weights, one would predict that DFCs — which route that structure out of the shared partition into A-exclusive — should spill *less*. Table 1 shows the predicted direction (5.9% DFC vs 8.9% CrossCoder) but the effect is modest and not significant. The top-five spillover runs (Table 2) include three CrossCoders with no exclusive partition at all and two DFCs with the largest exclusive share (10%): the relationship between exclusive-share and spillover is not monotone.

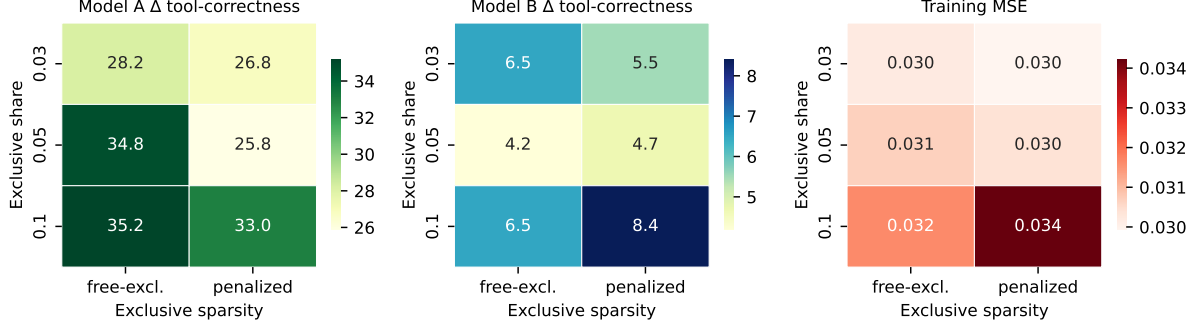


Figure 6: Configuration matrix for the DFC-only runs ($n=35$). **Left:** Model A fidelity (tool-correctness delta). **Center:** Model B spillover (tool-correctness delta). **Right:** training MSE. Penalizing the exclusive partition’s L1 tends to reduce Model A fidelity and has no consistent effect on Model B spillover.

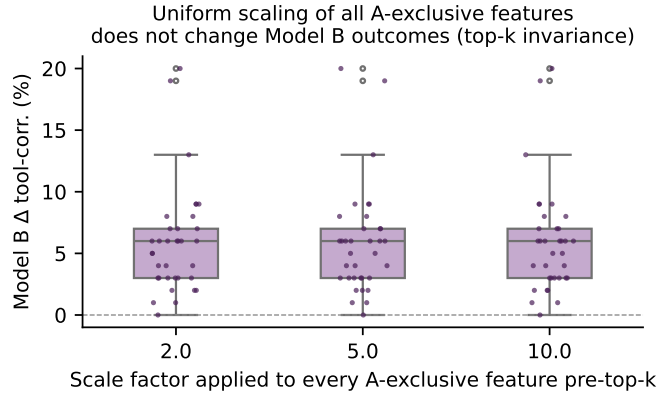


Figure 7: Uniform scaling of every A-exclusive feature is behaviorally inert. The three boxplots are the same configurations at $\alpha=2, 5$, and 10 ; every run gives identical outcomes at every scale. The distribution matches the Model-B Δ tool-correctness from the pure-reconstruction run (Sec. 5) to the decimal.

What spills? Format accuracy does not spill — base Model B post-reconstruction never produces the `<tool_call>` tag — but tool correctness does. This is consistent with the prior descriptive finding that A-20 and A-665 carry distinct roles: A-20 is tool-selection-and-argument-binding (the “what” of the call), A-665 is the surface convention for structured responses (the “how”). The spillover pattern suggests that the “what” channel has leaked into the shared partition during training while the “how” channel has not. This is a falsifiable prediction we will test with the targeted-neuron steering work described in Section 8.

L1 on the exclusive partition hurts A fidelity. Figure 6 shows that enforcing $\lambda_{\text{excl}}=10^{-3}$ on the exclusive partition (“penalized”) reduces Model-A fidelity relative to $\lambda_{\text{excl}}=0$ (“free-excl.”). The effect is consistent across the 5% and 10% exclusive shares. Sparsifying the exclusive partition appears to push more tool-specific signal back into the shared partition — the opposite of what the DFC design intends. Mechanistic tests on whether this moves specific features (e.g., whether A-20’s counterpart migrates partitions as λ_{excl} increases) are left to future work.

6 Naive Uniform Steering Is a No-Op

The natural baseline for a steering experiment on an exclusive-partition crosscoder is: scale all A-exclusive feature activations by a factor $\alpha>1$ before they enter Model B’s decoded activation. We ran this with $\alpha \in \{2, 5, 10\}$ across all 35 DFC configurations and with no individually selected neuron indices (all exclusive features uniformly scaled), reporting results in Figure 7.

All three scale factors produce identical outcomes: +0.118 overall delta, 0% format delta, +5.89% tool-correctness delta for every run, every scale. We verified that these equal the corresponding pure-reconstruction Model-B deltas from Section 5 to four decimal places. This is not a measurement artifact; it is a mathematical property of the top- k activation function used in the DFC. If $\mathbf{f}$ is the full pre-top- k feature vector and $\alpha\mathbf{f}_{\text{excl}}$ is the scaled subset, then scaling a contiguous fraction of $\mathbf{f}$ by a positive constant does not change *which* features rank in the top k : either each exclusive feature was already in the top- k (and scaling by α just rescales its post-decoder contribution *along with the corresponding scaling of the shared features that dominate the top- k — so the ratio is preserved*), or it was not (and scaling it by any bounded α may or may not push it above threshold). Empirically, the top- k selection is invariant across our scale range.

Implication. The obvious “amplify the exclusive partition” baseline for steering does not work as a standalone intervention under a top- k architecture. Any non-trivial steering requires either (i) post-top- k amplification targeted at specifically selected neurons, (ii) clamping targeted neurons to fixed values, or (iii) conditional clamping triggered on input properties. Our planned steering work (Sec. 8) consequently operates on hand-selected neuron subsets rather than the full exclusive partition.

7 Discussion

Reconciliation with the prior descriptive report. Our prior report, based on a single ($D=16,384, k=90, p=3\%$) DFC, identified A-20 and A-665 as the dominant tool-specific neurons and presented small reconstruction-improvement effects (+10 to +18% tool-correctness) as an incidental observation. The sweep analysis here reframes two of those findings. First, the reconstruction-improvement effect is not incidental — it is the norm across the sweep, with a mean of +31% on Model A. Second, the capability-spillover effect on Model B (then described as a “preview of the overstimulation hypothesis”) generalizes, but *not* in the direction the hypothesis predicted: more aggressive isolation of the capability into an exclusive partition does not produce more spillover. CrossCoders without any exclusive partition spill slightly more capability than DFCs with one.

What does this imply about the crosscoder assumption? The working conjecture behind using a two-model crosscoder instead of a one-model SAE is that the capability-specific features are less superpositioned with shared/base features when the decomposition is forced to distinguish the two models. Our sweep provides indirect evidence against the strong form of this conjecture. If the DFC’s exclusive partition cleanly contained the tool-calling capability, then (i) imposing L1 sparsity on it should not hurt Model-A fidelity (only redundant features should get pruned), and (ii) uniformly amplifying it should produce some steering. Neither holds. Tool-calling capability is smeared across the shared partition by design of the joint training, and the exclusive partition captures only the most model-specific residue. A meaningful fraction of the “what” of tool-calling lives in the shared partition.

Why format spillover is zero but tool spillover is not. A notable asymmetry: every one of the 47 sweep runs produces $\Delta\text{format accuracy}_B = 0$, yet most produce $\Delta\text{tool correctness}_B > 0$. This is possible because the evaluation rubric scores tool correctness based on whether the *intended* tool and arguments are present in the response — prose mentions of the correct function and argument values count — while format accuracy requires the exact `<tool.call>` structured tag. Semantic tool-calling intent transfers through the crosscoder; the surface convention does not. This is concordant with our prior interpretation that A-20 (structured `<tool.call>` JSON) and A-665 (structured-tag dialogue history) encode the surface-format channel, and that these

features are sufficiently model-A-specific that their contribution fails to leak into the shared partition during joint training.

8 Future Work

The sweep clarifies the two immediate next experiments required to close the capability-spillover and steering loops. Autointerp for shared-partition neurons and targeted-neuron steering results are pending; they will be slotted into the following subsections of the camera-ready version.

8.1 Autointerp of the Shared Partition

[Placeholder.] The prior descriptive report characterized only A-exclusive and B-exclusive neurons. Given the sweep finding that capability-relevant structure persists in the shared partition (Sec. 7), a targeted autointerp pass over shared neurons that co-activate with A-20 and A-665 under tool-use inputs is the natural next step. Expected deliverable: a ranked list of shared-partition neurons with tool-specific semantic roles and validation via the detection protocol of [Movva et al. \[2025\]](#).

8.2 Targeted-Neuron Steering

[Placeholder.] The null result of Section 6 establishes that uniform pre-top- k scaling is inert. We plan to compare three steering interventions applied to an individually selected neuron subset $S \subset \{\text{A-exclusive} \cup \text{tool-correlated shared}\}$: (i) post-top- k amplification by a coefficient, (ii) hard clamping to fixed activation values, (iii) conditional clamping gated on input features. Two neuron-selection algorithms will be tested: a naive “top- k most discriminative” heuristic, and a side-task-aware variant that excludes neurons with high activation on general-text inputs from FINEWEB. The cross-validation protocol is “design on one crosscoder configuration, reproduce on the other 46” to guard against overfitting to a single feature dictionary.

8.3 Knowledge Boundary and Stubbornness

[Placeholder.] Following [Pan et al. \[2026\]](#), once individual steerable neurons are identified, we will test whether modulation of tool-calling neurons also shifts the hesitation boundary (when does the model refuse to call a tool?) and stubbornness (how often does the model insist on its initial choice of tool despite evidence of failure?). Both are observable behavioral endpoints that do not require retraining.

9 Conclusion

We trained 48 crosscoder and DFC variants and evaluated 47 of them on 100 tool-use prompts each, producing a sweep-level characterization of reconstruction fidelity and capability spillover in crosscoder-based model diffing. Three results stand out. Reconstruction through a trained crosscoder improves — not merely preserves — the tool-calling behavior of the RL model, by a mean of +31% tool-correctness. Capability spillover to the frozen base model replicates across the sweep, but runs counter to the intuitive direction of the DFC design: the presence of a dedicated exclusive partition does not increase spillover, and CrossCoders without one spill modestly more capability on average. Uniform pre-top- k amplification of the exclusive partition is a mathematical no-op under our top- k architecture. The interpretability affordance of crosscoders for tool-use agents therefore appears to reside in identifying individual candidate neurons for targeted interventions, not in uniform manipulation of the labeled exclusive partition as a whole.

Limitations. (i) The sweep is a 48-point grid search, not a randomized trial; the Cross-Coder/DFC imbalance (12 vs 35) limits the power of the architecture comparison. (ii) All runs are at layer 13 of QWEN2.5-3B; layer selection is known to matter for crosscoder interpretability but we do not sweep it here. (iii) Our 100-prompt evaluation is a hold-out of a single task (TOOLRL); we do not test whether the spillover replicates on other RL-fine-tuned model pairs. (iv) Model-B format-accuracy spillover is identically zero, so we cannot study its distribution. (v) Autointerp and targeted-neuron steering are not yet slotted in; the camera-ready version will carry those results.

Reproducibility. Sweep metrics are logged at <https://wandb.ai/standartikom-northwestern-university/dfc-crosscoder-sweep>. All 47 behavioral evaluations will be released alongside the camera-ready paper.

References

- Emre Can Acikgoz et al. ToolRL: Reward is all tool learning needs. *arXiv preprint*, 2025a.
- Emre Can Acikgoz et al. ToolRL dataset. <https://huggingface.co/datasets/emreacanacikgoz/ToolRL>, 2025b.
- Trenton Bricken, Adly Templeton, Joshua Batson, et al. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023.
- Thomas Jiralerspong and Trenton Bricken. Model diffing without borders: Unlocking cross-architecture model diffing. *Mechanistic Interpretability Workshop at NeurIPS 2025*, 2025.
- Jack Lindsey, Adly Templeton, Jonathan Marcus, Thomas Conerly, Joshua Batson, and Christopher Olah. Sparse crosscoders for cross-layer features and model diffing. *Transformer Circuits*, 2024.
- Rajiv Movva et al. Validation methods for neural network interpretability. *arXiv:2502.04382*, 2025.
- Long Ouyang, Jeffrey Wu, Xu Jiang, et al. Training language models to follow instructions with human feedback. *NeurIPS*, 2022.
- Xu Pan, Jingxuan Fan, Zidi Xiong, Ely Hahami, Jorin Overwiening, and Ziqian Xie. User-assistant bias in LLMs. <https://arxiv.org/abs/2508.15815>, 2026.
- Qwen Team. Qwen2.5 technical report. *arXiv:2412.15115*, 2025.
- Adly Templeton, Tom Conerly, Jonathan Marcus, et al. Scaling monosemanticity. *Transformer Circuits Thread*, 2024.